

Contexto del proyecto: Análisis de Pliegos de Licitación

Documento de referencia para usar como contexto en distintas etapas de BMAD Method — brief, PRD, UX, arquitectura e historias. Contiene los hechos estables del proyecto que no cambian entre etapas.

1 Qué es este documento

No es una skill ni un workflow — es **material de referencia** para pegar o resumir al arrancar cualquiera de las etapas de BMAD. Cada sección indica en qué etapa conviene usarla:

Brief **PRD** **UX** **Arquitectura** **Historias**

2 El problema y el producto

Brief **PRD**

Problema: el equipo participa en licitaciones y necesita revisar pliegos extensos y heterogéneos para decidir si conviene participar. Hacerlo manualmente es lento y propenso a errores u omisiones (plazos, garantías, causales de rechazo).

Qué hace el sistema: analiza pliegos de licitación en PDF y extrae automáticamente 7 categorías:

1. Objeto y alcance
2. Requisitos de admisibilidad
3. Garantías
4. Plazos clave
5. Criterios de evaluación
6. Causales de rechazo
7. Anexos obligatorios

La salida queda en JSON estructurado (consultas exactas) y vectorizada en Azure AI Search (búsqueda semántica).

Usuarios: el propio equipo, uso interno por ahora — no es un producto público todavía.

3 Alcance del MVP

Brief **PRD** **Historias**

Incluye

- Subida manual de un pliego en PDF (mezcla de pliegos nativos y escaneados)
- Extracción de las 7 categorías
- Frontend con 3 pantallas: Dashboard, Análisis de Pliegos, Historial
- Pipeline: ingesta → chunking → 7 nodos de extracción en paralelo → merge/validación → storage

No incluye (por ahora)

- Scoring o puntaje por pliego — no está definido qué mediría a nivel de negocio
- "Radar de oportunidades" — sección a cargo de otro integrante del equipo, reservada en el sidebar pero no se construye en este MVP
- El frontend **todavía no está construido** — hay capturas de referencia, pero son solo inspiración de estilo, no pantallas ya hechas

4 Stack tecnológico

Arquitectura **Historias**

Componente	Herramienta
Lenguaje	Python 3.11+
Ingesta de documentos	Azure AI Document Intelligence (modelo <code>prebuilt-layout</code>)
Vector store + búsqueda + metadata	Azure AI Search (índice único: texto, vector, campos filtrables)
Modelos de IA (chat + embeddings)	Azure OpenAI
Orquestación	LangGraph (Microsoft Agent Framework descartado por ahora)
Validación de datos	Pydantic
Planificación	BMAD Method (sin Spec Kit / SDD)
Despliegue	Azure Container Apps

POR QUÉ AZURE AI SEARCH REEMPLAZA DOS PIEZAS

Un mismo índice guarda texto buscable, vector de embedding, y campos filtrables (organismo, fecha, estado) — no hace falta un vector store separado de una base para el JSON estructurado.

5 Arquitectura del código

Arquitectura Historias

Patrón: capas simples + adapters livianos (no arquitectura hexagonal completa). Una clase por servicio externo de Azure, sin interfaz formal; un archivo por nodo del grafo.

```
src/
├── schemas.py                # Pydantic: PliegoExtraido + 7 categorías
├── adapters/                 # una clase por servicio externo (Azure), sin interfaz formal
│   ├── document_intelligence.py
│   ├── ai_search.py
│   └── openai_client.py
├── nodes/                    # un archivo por nodo – unidad natural de cada historia
│   ├── ingesta.py
│   ├── chunking.py
│   └── extraccion/
│       ├── objeto_alcance.py
│       ├── requisitos_admisibilidad.py
│       ├── garantias.py
│       ├── plazos_clave.py
│       ├── criterios_evaluacion.py
│       ├── causales_rechazo.py
│       └── anexos_obligatorios.py
│   ├── merge_validacion.py
│   └── storage.py
├── graph.py
├── api/
│   └── routes.py
```

Por qué esta y no hexagonal completa: con Azure ya fijo (sin migración de nube pendiente) y BMAD como única metodología, la arquitectura que más rinde es la que se describe fácil en `bmad-architecture` y mapea 1 a 1 con `bmad-create-epics-and-stories`. Los adapters siguen siendo mockeables en tests (sin gastar créditos reales), pero sin la ceremonia de puertos formales.

6 Los 7 nodos de extracción — principio de diseño

Arquitectura Historias

Cada categoría es un nodo independiente (no un solo nodo monolítico), corren en paralelo, y convergen en un nodo de merge y validación que arma el JSON final.

Por qué están separados: paralelización real, iteración aislada (ajustar un nodo sin tocar los otros 6), y cada uno es una historia de BMAD acotada y una unidad de test independiente.

Extracción robusta ante variabilidad de redacción (los pliegos no usan siempre el mismo vocabulario — ej. "plazo de presentación" puede aparecer como "fecha de apertura de sobres"):

- El prompt describe el *significado* del campo, no pide una frase literal
- Glosario de sinónimos por categoría, armado con pliegos reales
- Hybrid search de Azure AI Search (vector + keyword) para encontrar los chunks candidatos, no matching literal
- Cada campo devuelve evidencia (cita del chunk) + nivel de confianza; confianza baja → revisión humana, nunca se inventa un valor
- Golden set de 15-20 pliegos anotados a mano para medir calidad de extracción

7 Frontend — diseño de referencia

UX PRD Historias

Pantalla	Función
Dashboard	Resumen: pliegos analizados, último análisis, tabla de últimos análisis (pliego, organismo, fecha, estado)
Análisis de Pliegos → Nuevo	Carga del PDF (drag & drop o selector) + botón "Analizar con IA"
Historial	Listado completo, búsqueda por pliego u organismo, acciones (ver, duplicar, eliminar)

Sidebar pensado para escalar: separa "Plataforma" (lo que se construye en este MVP) de "Próximamente" (Radar de oportunidades, Chat IA, CRM, Calendario) — agregar una sección nueva es sumar un ítem, no reestructurar.

8 Metodología y equipo

Brief

- Planificación y desarrollo 100% con **BMAD Method** (sin Spec Kit / SDD)
- Herramienta de trabajo: **GitHub Copilot Chat** en VS Code
- Ciclo por historia: `bmad-create-story` → `bmad-dev-story` → `bmad-code-review` , siempre el mismo, sin excepciones
- Ramas: `main` (protegida) / `develop` (integración) / `feature/<historia>` y `fix/<algo>` por historia
- Equipo chico, mixto — recién arrancando con estas herramientas puntuales (BMAD, LangGraph en un proyecto real)
- Un integrante adicional se va a encargar más adelante de "Radar de oportunidades"

9 Estimación

Brief PRD

~13,5-19,5 días hábiles (2,5-4 semanas) para el MVP completo, incluyendo diseño y construcción del frontend, setup de recursos de Azure, los 7 nodos de extracción y el nodo de merge.

10 Decisiones abiertas (no resolver todavía)

Brief PRD

- Qué mide el scoring por pliego (a definir con feedback real de uso, no antes)
- Tier definitivo de Azure AI Search en producción (depende del volumen real)
- Modelo de embeddings y de chat definitivos (a validar contra el golden set)

NOTA SOBRE FUTURAS RONDAS DE PLANIFICACIÓN
Cuando se sume alcance nuevo (Radar de oportunidades, scoring), no se rehace el brief — se usa `bmad-prd` en modo **Update** sobre el PRD existente. El brief y la arquitectura de base se mantienen; solo se actualizan con el alcance nuevo.

